

IT7075 · APPLIED AI FOR CYBERSECURITY · MODULE 4

Retrieval-Augmented Generation

Ground a model in knowledge you trust — then build the whole pipeline yourself, with no framework hiding the moving parts.

Six parts · two notebooks · a concept-check lab and a mini-project

Framework-free, and offline by default

PART 1

The Problem & the Core Idea

What goes wrong when a model answers from memory, and the two-phase pipeline that fixes it.

MODULE MAP

- **The Problem & the Core Idea**
- Embeddings
- Vector DB & Chunking
- The RAG Recipe & Trust
- Build It: Notebooks & Practice
- Lab & Mini-Project

The problem: it answers anyway

A model asked something outside its knowledge does not decline. It produces something plausible.

Asked about a corpus it has never seen

```
> What does our incident response runbook say about  
   isolating a compromised domain controller?
```

```
"Your runbook specifies immediate network isolation,  
  followed by a forensic image before any reboot..."
```

```
# It has never seen your runbook.
```

And you cannot simply paste it all in. The context window is finite, you pay per token, and retrieval quality degrades as you bury the relevant passage in irrelevant ones.

The fix, in one sentence: look up the relevant passages first, hand them to the model, and instruct it to answer only from those. The model stops being a source of facts and becomes a summariser of facts you supplied — which is a much more reliable thing to be.

It invents

Confident, well-formed, and unsupported by anything.

It is stale

A training cutoff. This week's CVEs do not exist to it.

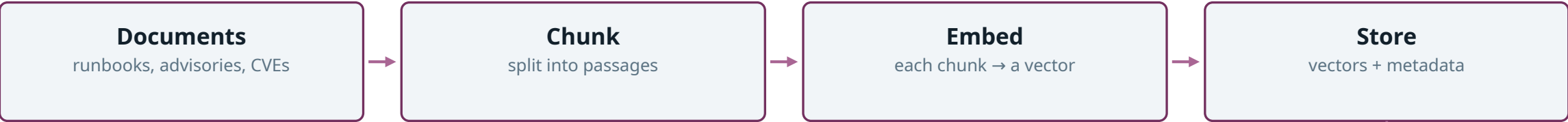
It is not yours

It knows the public internet, not your policies or assets.

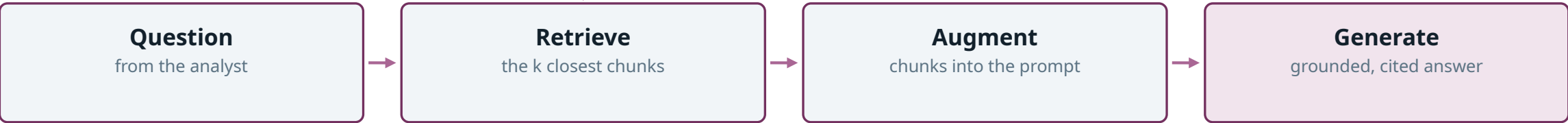
What RAG is: two phases, not one

Build the store once. Query it many times. Confusing the two is the most common early mistake.

INGESTION — YOU RUN THIS ONCE, WHEN DOCUMENTS CHANGE



QUERY — YOU RUN THIS ON EVERY QUESTION



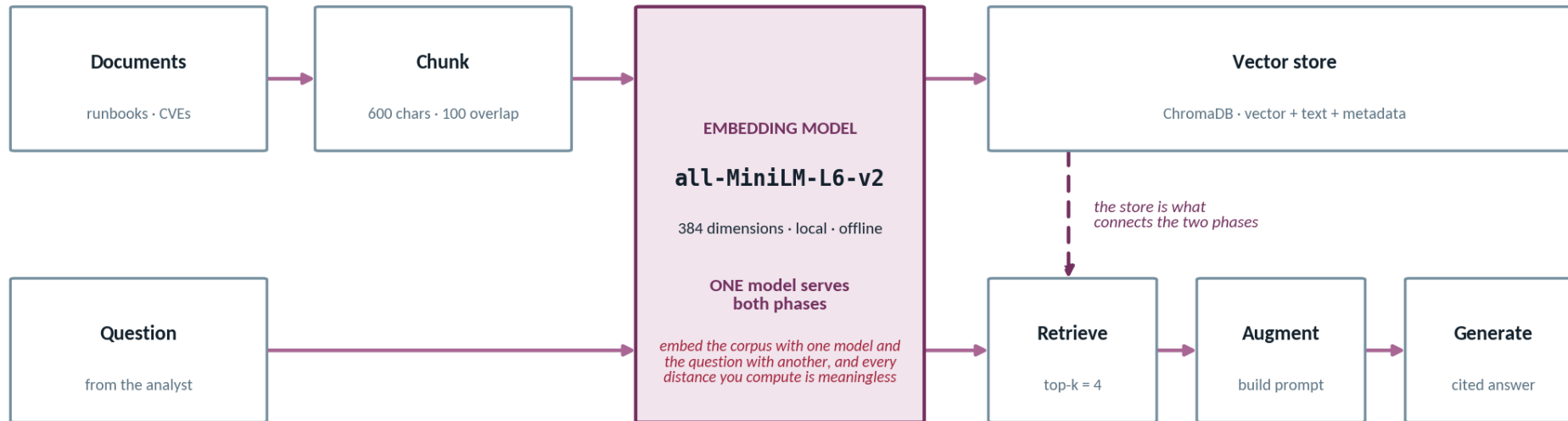
Phase	When it runs	What it costs	What breaks if you get it wrong
Ingestion	Once, then again only when documents change	Slow and compute-heavy — every chunk gets embedded	Re-embedding on every question: correct answers, unusable latency
Query	Every single question	Fast — one embedding, one nearest-neighbour search	Retrieving too few chunks, or too many irrelevant ones

The retrieved passages are the evidence. They are what the answer is built from, and what a citation points at.

Two phases, one embedding model

Build the store once. Query it many times. Both sides must be turned into vectors by the same model.

INGESTION • run once, whenever the documents change



QUERY • run on every single question

INGESTION — lecture3_rag_pipeline.ipynb, STEP 2

```
chunks = split_into_chunks(text)
collection.add(ids=ids, documents=chunks,
               embeddings=embed(chunks),
               metadatas=labels)
```

QUERY — lecture3_rag_pipeline.ipynb, STEP 3 • note embed() on both sides

```
def retrieve(query, how_many=4, max_distance=0.75):
    results = collection.query(
        query_embeddings=embed([query]),
        n_results=how_many)
```

PART 2

Embeddings — Search by Meaning

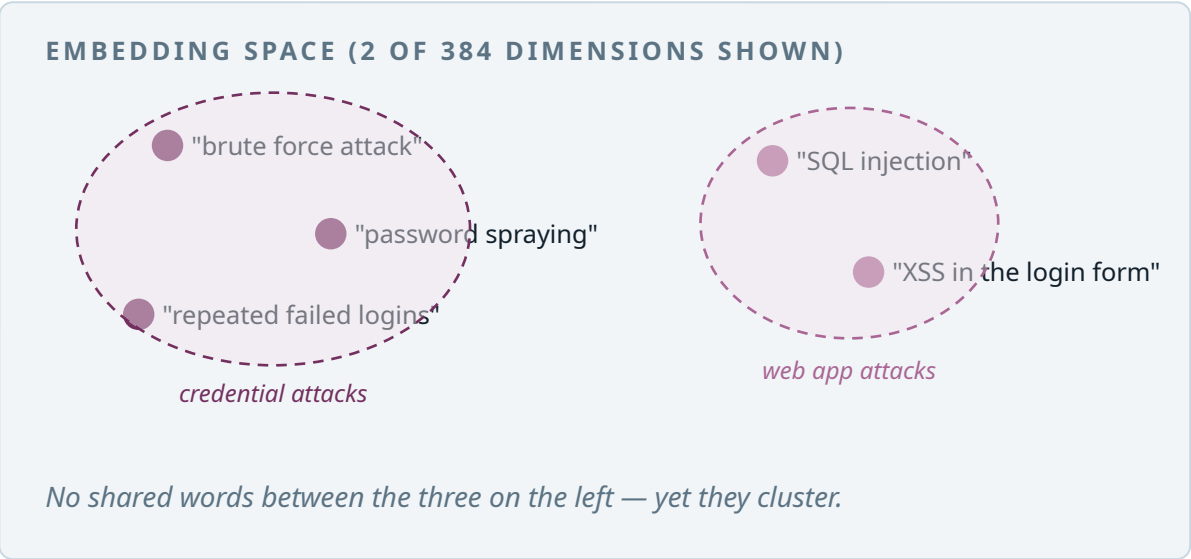
The engine under retrieval: turning text into vectors so that closeness in space means closeness in meaning.

MODULE MAP

- The Problem & the Core Idea
- **Embeddings**
- Vector DB & Chunking
- The RAG Recipe & Trust
- Build It: Notebooks & Practice
- Lab & Mini-Project

Embeddings turn meaning into geometry

Text becomes a point in space. Related text lands nearby. That is the whole trick.



Local, offline, nothing leaves your machine

```
from sentence_transformers import SentenceTransformer
m = SentenceTransformer("all-MiniLM-L6-v2") # 384 dims, ~80 MB
vecs = m.encode(["brute force attack", "password spraying"])
```

Term	What it means here
Embedding	A fixed-length list of numbers standing for a piece of text
Dimensions	How long that list is. 384 for the small model we use
Cosine similarity	The angle between two vectors. 1 = same direction, 0 = unrelated
Embedding model	The thing that does the mapping. Different from your chat model

Why this matters for security work: the embedding model is small enough to run on a laptop CPU. Your corpus — runbooks, incident notes, advisories — never leaves your machine to be turned into vectors. That is a genuine privacy property, and it is why we default to local embeddings rather than an embedding API.

Embeddings in twenty lines

The whole of section 2 of `lecture2_embeddings_and_vectordb.ipynb` — nothing hidden, no framework.

lecture2_embeddings_and_vectordb.ipynb · SECTION 2 — COMPLETE

```
from sentence_transformers import SentenceTransformer
import numpy as np

# a small, free embedding model that runs on your own computer
model = SentenceTransformer("all-MiniLM-L6-v2")

phrases = ["SQL injection in the login form",
           "command injection via crafted input",
           "weak TLS cipher configuration"]

# turn each phrase into a vector
vectors = model.encode(phrases)
print("Each phrase became a list of", len(vectors[0]), "numbers.")

# how similar are two vectors? 1.0 = same meaning, 0 = unrelated
def similarity(vector_a, vector_b):
    dot = np.dot(vector_a, vector_b)
    sizes = np.linalg.norm(vector_a) * np.linalg.norm(vector_b)
    return float(dot / sizes)

for i in range(len(phrases)):
    for j in range(i + 1, len(phrases)):
        print(round(similarity(vectors[i], vectors[j]), 3),
              phrases[i], "<->", phrases[j])
```

1 One model, downloaded once

~90 MB on first use, then cached. The first run looks frozen — it is downloading, not hanging.

2 `encode()` takes a list

Returns one row of 384 numbers per phrase. That is the entire embedding step: one function call.

3 Cosine, written out

Dot product over the product of the two lengths. Three lines of NumPy — no framework anywhere.

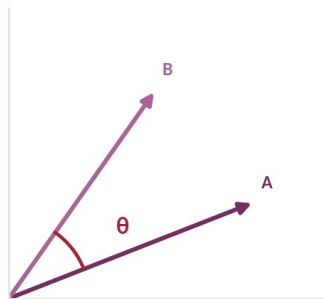
4 What you should see

The two injection phrases score ~0.5–0.6 despite sharing almost no words. TLS scores far lower. That is semantic search.

Three ways to measure closeness

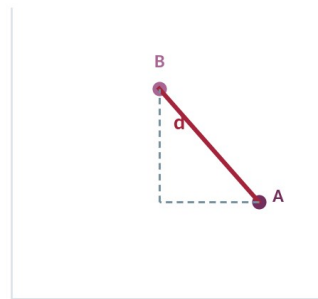
Cosine is the default for text. Know the other two — and know which one ChromaDB uses when you do not say.

Cosine similarity



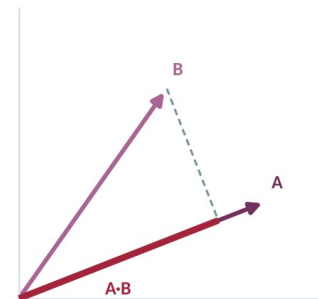
the angle only — length is ignored
 $\cos \theta = 1$ identical · 0 unrelated

Euclidean distance



straight-line gap — Pythagoras
 0 identical · sensitive to length

Dot product



angle and length together
 on normalised vectors $\equiv$ cosine

COSINE BY HAND — lecture2_embeddings_and_vectordb.ipynb, SECTION 2

```
def similarity(vector_a, vector_b):
    dot = np.dot(vector_a, vector_b)
    sizes = np.linalg.norm(vector_a) * np.linalg.norm(vector_b)
    return float(dot / sizes)
```

SETTING THE METRIC — lecture3_rag_pipeline.ipynb, STEP 2

```
collection = client.get_or_create_collection(
    "security_kb", metadata={"hnsw:space": "cosine"})

# omit the metadata and Chroma silently uses L2
```

The same three matches, two different scales. Lecture 2 with the default L2 reports 0.70 / 0.78 / 0.82. Lecture 3 with cosine reports 0.307 / 0.366 / 0.386. Retrieval did not improve — a threshold tuned on one scale is meaningless on the other.

Why embeddings beat keyword search

Security language is varied and paraphrased. Exact-string matching is the wrong tool for it.

KEYWORD SEARCH

Query: "brute force"

- "password spraying detected" — no match
- "repeated authentication failures" — no match
- "brute force protection enabled" — matched, and irrelevant

SEMANTIC SEARCH

Query: "brute force"

- "password spraying detected" — 0.71
- "repeated authentication failures" — 0.68
- "brute force protection enabled" — 0.44, ranked lower

	Keyword (exact string)	Semantic (meaning)
Finds paraphrase	No — different words, no match	Yes — that is the entire point
Handles a typo or an acronym	Poorly	Usually, if the meaning survives
Explains why it matched	Trivially — you can see the word	Less so — a similarity score is not a reason
Still worth having	Yes, for exact IDs: CVE numbers, hostnames, hashes	Yes, for everything written in prose

Choosing an embedding model

MTEB ranks models across dozens of retrieval, clustering and classification tasks. Bigger is not automatically better.

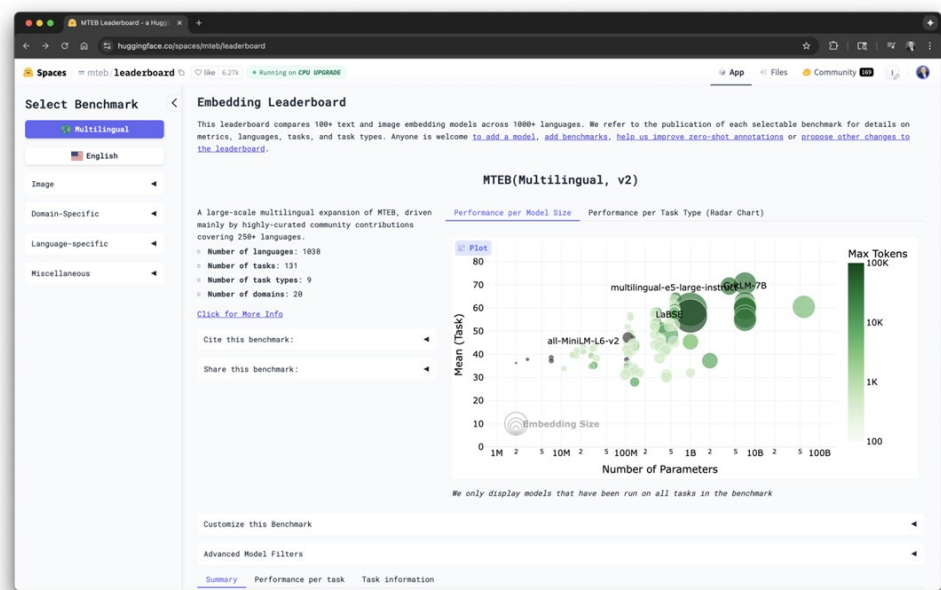


Figure 2-2 from Santos, Agentic AI for Cybersecurity — reproduced for classroom use.

```
THE OPTIONAL CELL — lecture2_embeddings_and_vectoradb.ipynb, SECTION 2

r = OpenAI().embeddings.create(input=text,
                                model="text-embedding-3-small")
len(r.data[0].embedding)    # -> 1536
```

Model	Dims	Notes
all-MiniLM-L6-v2	384	local · offline · ~90 MB · our default
text-embedding-3-small	1536	OpenAI · API key · low cost
text-embedding-3-large	3072	higher accuracy · higher cost
text-embedding-ada-002	1536	legacy, still widely deployed

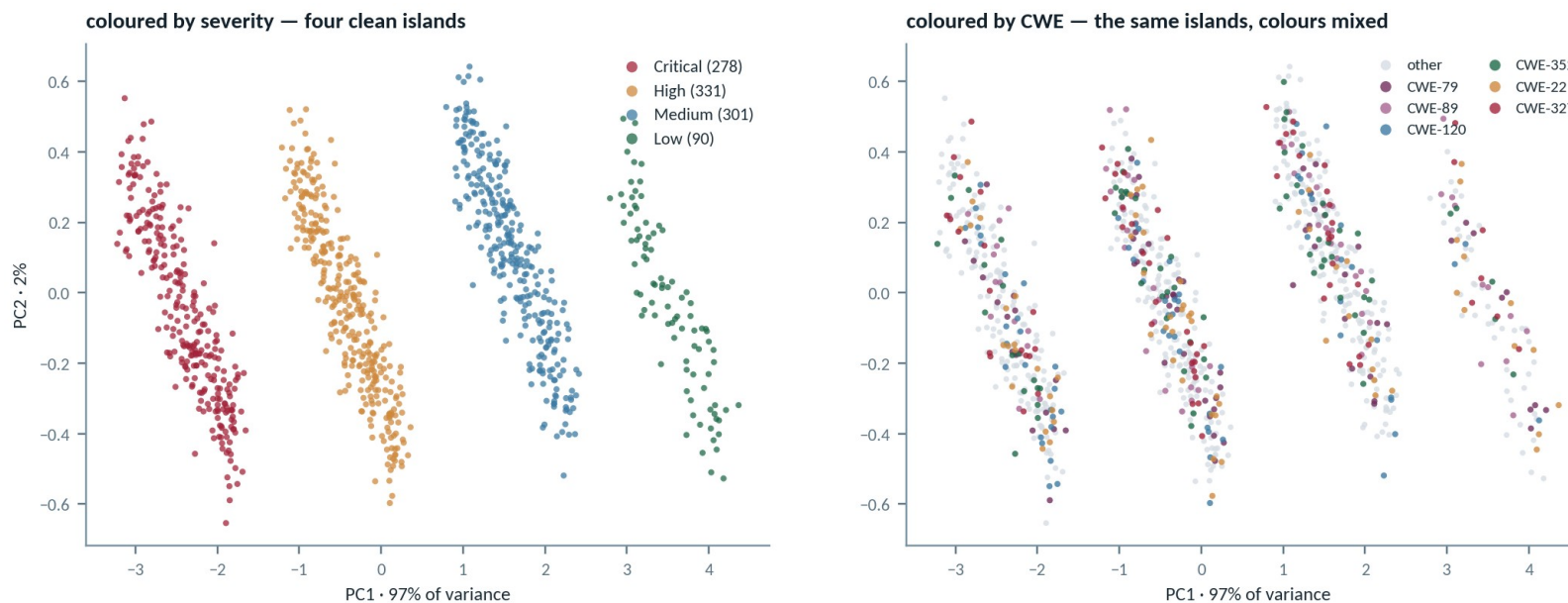
More dimensions is not free. Every extra dimension costs storage, memory and query time — permanently. A 384-d model that runs on a laptop CPU with no network is the right default for security work: your corpus never leaves the machine to be vectorised.

What actually decides it

Your data and your task — not the leaderboard position. Shortlist from MTEB, then benchmark two candidates on your own questions before you commit.

The meaning-space map

1,000 CVEs projected down to two dimensions — the plot the notebook cannot draw when the matplotlib backend is blocked.



1,000 CVEs · PCA of the shipped 8-D vectors. Severity separates; the vulnerability class does not — which is why nearest-neighbour search on these vectors returns unrelated CVEs.

lecture2_embeddings_and_vectordb.ipynb, SECTION 7 — THE CELL THAT OFTEN CANNOT RENDER

```
from sklearn.decomposition import PCA

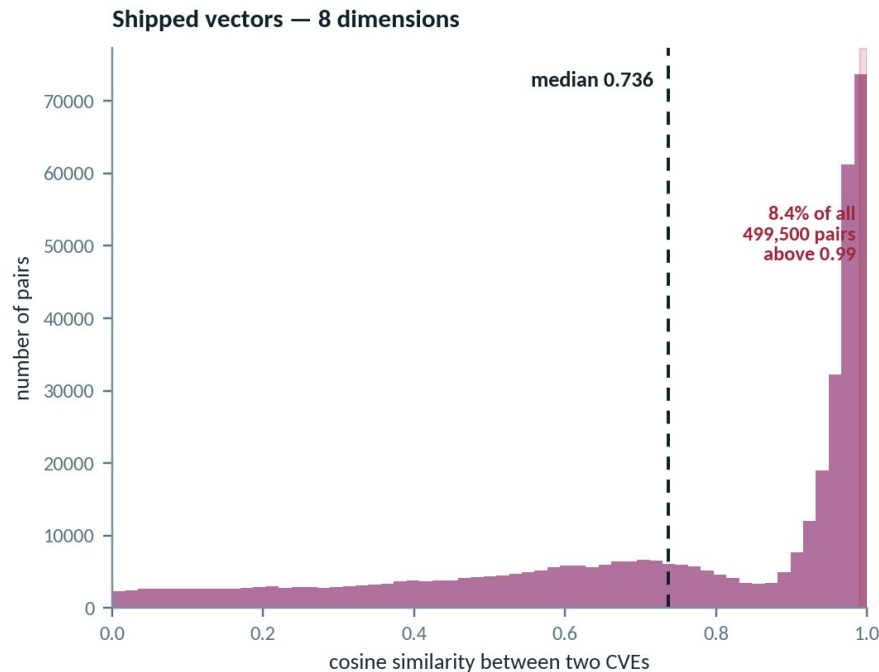
# squeeze each CVE vector down to 2 numbers so we can plot it
flat = PCA(n_components=2).fit_transform(cve_vectors)
```

What you are looking for

Structure. Severity separates cleanly; vulnerability class does not. If the vectors encoded what the CVE says, CWE would cluster too — and it does not.

When embeddings collapse

The vectors shipped with this module have 8 dimensions, not 384. Here is exactly what that does to retrieval.



What the neighbours actually look like

8-D · query = CVE-2026-0001, CWE-79 (XSS) 0 / 10 match

CVE-2027-0038	0.999	CWE-451
CVE-2026-0049	0.999	CWE-77
CVE-2027-0133	0.999	CWE-269

384-D MiniLM · query = a CWE-918 (SSRF) entry 8 / 10 match

CVE-2026-0061	1.000	CWE-918
CVE-2030-0023	0.990	CWE-918
CVE-2028-0042	0.990	CWE-918

same-CWE hits in the top 10 · chance level $\approx 0.7 / 10$ · 384-D figures measured in the instructor key

lecture2_embeddings_and_vectoradb.ipynb, SECTION 3 — ADD ONE LINE AND THE MYSTERY DISSOLVES

```
cve_vectors = np.loadtxt("cve_vectors_1000.tsv", delimiter="\t")
cve_vectors.shape      # -> (1000, 8)  not (1000, 384)

for row in range(len(cve_vectors)): # section 3, unchanged
    scored.append((similarity(cve_vectors[0], cve_vectors[row]), row))
```

“The neighbours look unrelated” is the right answer

Report it, and say why. Re-embedding the descriptions with MiniLM and showing the difference goes beyond the rubric.

PART 3

Store It: Vector DB, Indexing & Chunking

Once text is vectors, you need somewhere to keep them, a way to find the nearest ones fast, and a decision about how big a passage should be.

MODULE MAP

- The Problem & the Core Idea
- Embeddings
- **Vector DB & Chunking**
- The RAG Recipe & Trust
- Build It: Notebooks & Practice
- Lab & Mini-Project

Vector databases and indexing

A store that keeps the vector, the text, and where it came from — and finds neighbours fast.

ONE STORED RECORD

Vector [0.021, -0.114, 0.335, ...] · 384 floats

Document the chunk text itself, returned verbatim

Metadata source file · section · date · classification

ID runbook.md#chunk-14

ChromaDB — and the NumPy equivalent, for perspective

```
import chromadb
col = chromadb.Client().create_collection("cves")
col.add(ids=ids, embeddings=vecs, documents=texts, metadatas=meta)
hits = col.query(query_embeddings=[qv], n_results=4)
```

```
# what it is doing, for a few hundred chunks:
import numpy as np
sims = (M @ qv) / (np.linalg.norm(M, axis=1) * np.linalg.norm(qv))
top = np.argsort(-sims)[:4]
```

Store	When it is the right choice
NumPy array	A few hundred chunks. Transparent, no dependency, fine
ChromaDB	This course. Metadata filtering and persistence, minimal setup
FAISS	Millions of vectors, one machine, you want speed
pgvector	You already run Postgres and want one system to operate

What indexing buys you: approximate nearest-neighbour search, so you do not compare the query against every vector. It trades a little recall for a lot of speed.

At our scale you do not need it. Understand it, because production corpora do.

Operating ChromaDB: four calls

Section 5 of lecture2_embeddings_and_vectordb.ipynb. Create, add, query, filter — and one commented line that makes it survive a restart.

lecture2_embeddings_and_vectordb.ipynb · SECTION 5 — COMPLETE

```
import chromadb

client = chromadb.EphemeralClient()          # lives in memory
collection = client.get_or_create_collection("ssrf_kb")

chunk_vectors = model.encode(chunks)
ids = ["chunk-" + str(i) for i in range(len(chunks))]
labels = [{"source": "ssrf.txt"} for _ in chunks]

collection.add(ids=ids,
               documents=chunks,
               embeddings=chunk_vectors.tolist(),
               metadatas=labels)
print("The database now holds", collection.count(), "chunks.")

question_vector = model.encode(
    "What is SSRF and how is it exploited?").tolist()
results = collection.query(query_embeddings=[question_vector],
                           n_results=3)

# filter on the labels you stored earlier
filtered = collection.query(query_embeddings=[question_vector],
                           n_results=2,
                           where={"source": "ssrf.txt"})

# to survive a restart, swap one line:
# client = chromadb.PersistentClient(path="my_db_folder")
```

1 Create — no server involved

EphemeralClient runs inside the notebook process. Nothing to install, nothing to start, no port.

2 Add — four parallel lists

ids, documents, embeddings, metadatas: same length, same order. That is the only real invariant in the API.

3 Query — mind the [0]

Results are wrapped in an outer list because you may ask several questions at once. Hence results["documents"][0].

4 Filter — metadata earns its keep

where={...} matches what you stored. Restrict by source, classification or date — this is how access control starts.

The vector database landscape

Ten systems you will meet. The choice turns on scale, deployment model, and what your team already runs.

Database	Type	Key characteristics / best for
Chroma	Open source	Lightweight and developer-friendly. Local prototyping and small-to-medium RAG — what we use.
FAISS (Meta)	Library	Not a database: the similarity-search library underneath many other systems. Custom infrastructure.
Milvus	Open source	Billion-scale vector search, highly configurable. Large enterprise deployments needing fine control.
Qdrant	Open source	Written in Rust. Advanced filtering and payload management. Performance-critical self-hosted work.
Weaviate	Open source	AI-native with built-in vectorisation modules and GraphQL. Multi-modal search and knowledge graphs.
pgvector	Postgres extension	Adds vector search to an existing PostgreSQL database. The obvious pick if you already run Postgres.
Pinecone	Managed service	Serverless and fully managed. Teams that want to ship without operating infrastructure.
MongoDB Atlas	General-purpose DB	Integrated vector search inside the document database. Natural for existing Mongo applications.
Vespa	Open source	Low-latency serving of search, ranking and recommendation together, including vectors.
AWS S3 Vectors	Object storage	Cheap durable storage for very large vector sets, loaded into a search library for querying.

THE ENTIRE SETUP — `lecture2_embeddings_and_vectordb.ipynb`, SECTION 5

```
client = chromadb.EphemeralClient()          # in memory, no server
# client = chromadb.PersistentClient(path="my_database_folder")
collection = client.get_or_create_collection("ssrf_kb")
```

Adapted from textbook Table 2-1

Santos, Agentic AI for Cybersecurity. We use Chroma because it runs in-process; the concepts transfer unchanged.

Why the database is fast

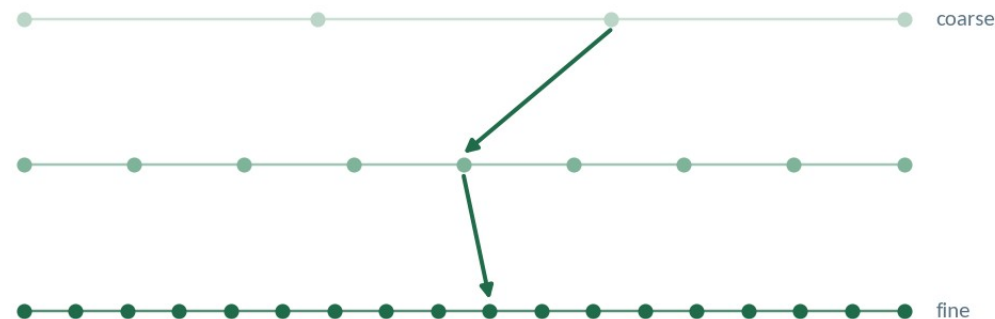
You wrote the brute-force loop yourself. A vector database builds an approximate nearest-neighbour index instead.

BRUTE FORCE • the loop you wrote by hand



1,000 comparisons • 11.5 ms • grows linearly with N

ANN INDEX (HNSW) • what ChromaDB builds for you



a handful of hops • scales to billions of vectors

lecture2_embeddings_and_vectoradb.ipynb, SECTION 6 — THE WHOLE COMPARISON

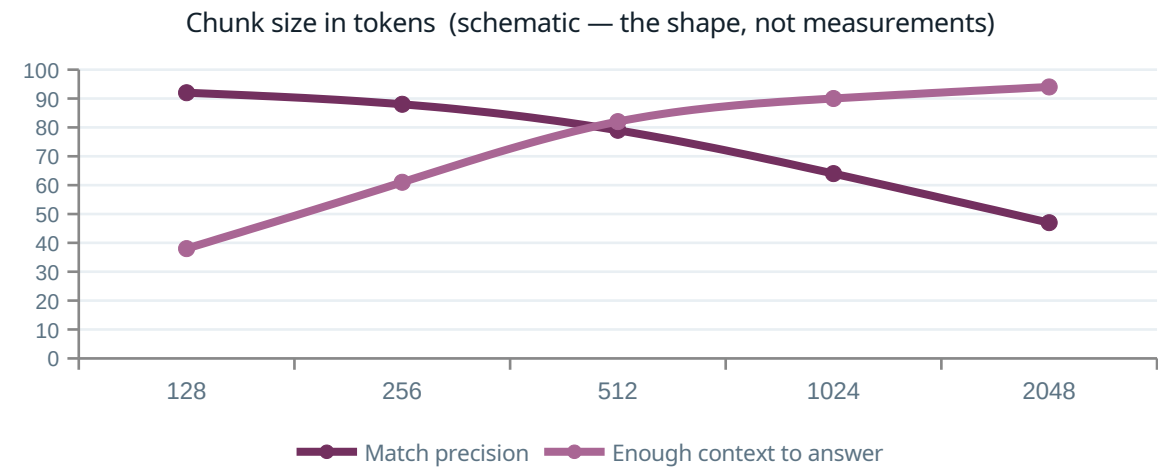
```
start_time = time.time()
hand_scores = []
for row in range(len(cve_vectors)):
    hand_scores.append(similarity(cve_vectors[0], cve_vectors[row]))
elapsed_ms = (time.time() - start_time) * 1000    # -> 11.5 ms
```

At our scale you do not need an index

Eleven milliseconds over a thousand vectors is fine — do not add one to the mini-project. Understand HNSW anyway: a production corpus is millions of chunks, and a linear scan there is not a slower answer, it is no answer.

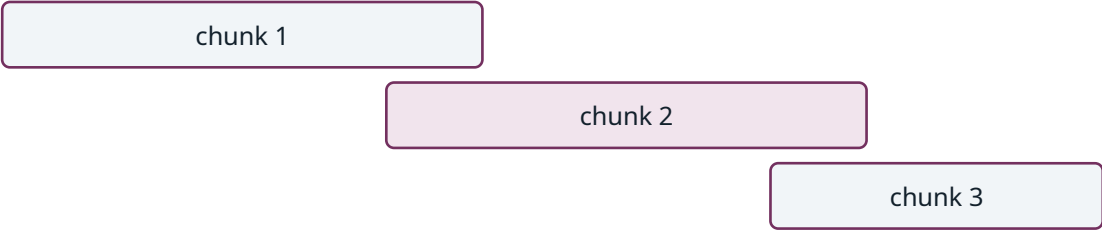
Chunking: the one knob that really matters

Too big and the match is diluted. Too small and the passage cannot answer the question.



Symptom	Usually means
Right topic, wrong detail	Chunks too large — the answer is buried in the passage
Fragmentary, unusable hits	Chunks too small — no context survived the split
"I don't know" too often	Too few chunks retrieved, or the threshold is too strict

OVERLAP KEEPS AN IDEA FROM BEING CUT IN HALF



the shaded overlap appears in both neighbours

A sensible default for security prose

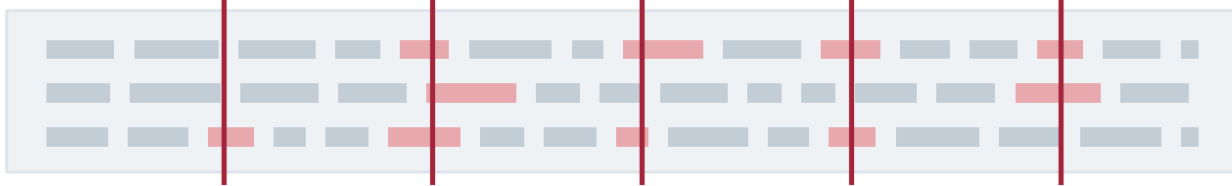
```
chunk_size = 512      # tokens
chunk_overlap = 64    # ~12% — enough to bridge a boundary
```

This is the mini-project's tuning experiment. Change the size, measure what happens, and report it. "512 because a tutorial said so" is not an answer.

Chunking: where the cuts land

Size is not the only decision. The boundary determines whether a retrieved chunk can answer anything at all.

FIXED 600-CHARACTER SLICES • the shipped splitter

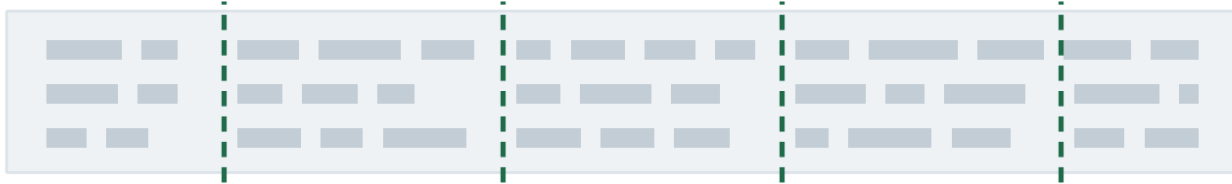


top-1 chunk begins:

"ed to perform an SSRF
attack. Take a look..."

18 of 22 chunks start mid-word

PACK WHOLE PARAGRAPHS, THEN CAP AT 600



top-1 chunk begins:

"This cheat sheet will focus
on the defensive point..."

0 of 24 chunks start mid-word

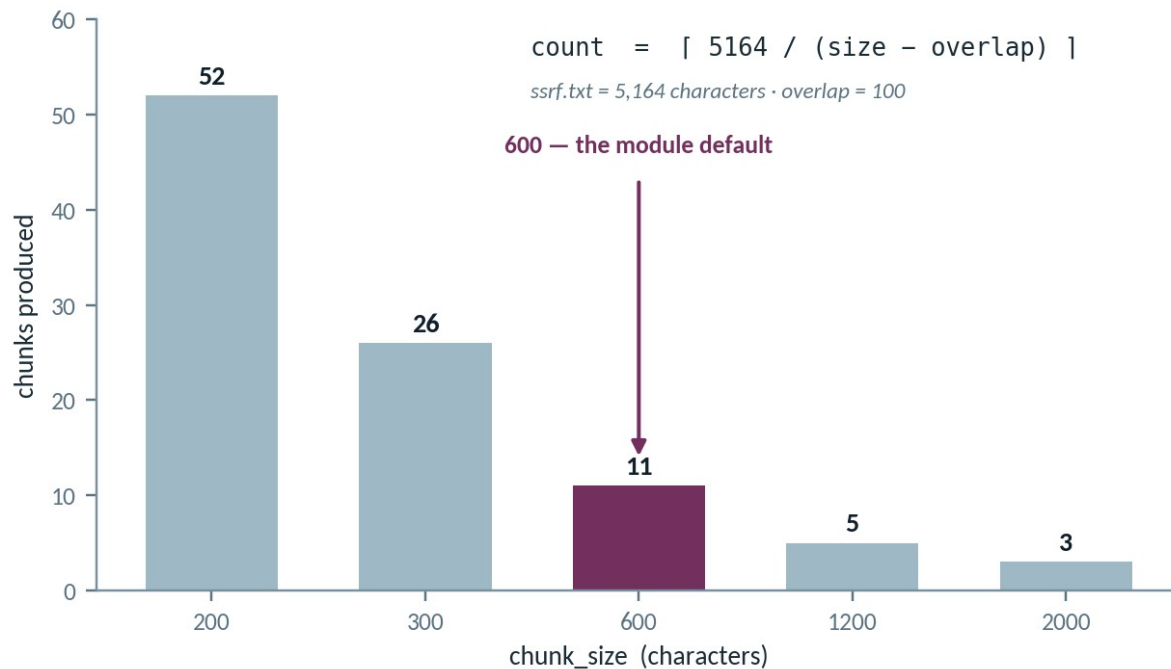
The retrieval score barely moves — 0.350 → 0.325. What changes is whether the retrieved passage can actually answer the question: 4/5 → 5/5 at k=1. The distance did not tell you the chunks were broken; reading them did.

lecture2_embeddings_and_vectordb.ipynb, SECTION 4 — CHARACTER COUNTING, NOTHING MORE

```
while start < len(document):
    piece = document[start : start + chunk_size].strip() # 600 characters, blind to words
    if piece != "": chunks.append(piece)
    start = start + chunk_size - overlap                # advance 500, take the next 600
```

Chunk size, chunk count — and one trap

The count follows from arithmetic. The trap hangs your notebook with no error message at all.



lecture3_rag_pipeline.ipynb, STEP 1 — NOTE WHAT GUARDS THE STRIDE

```
def split_into_chunks(text, chunk_size=600, overlap=100):
    pieces, start = [], 0
    while start < len(text):
        pieces.append(text[start : start + chunk_size])
        start = start + chunk_size - overlap
```

overlap ≥ chunk_size → the cell never finishes

The stride is $chunk_size - overlap$. Set overlap to 600 with a chunk_size of 600 and the stride is zero: start never advances and the loop runs forever. No exception, no output. A hung cell in this module is almost always this.

TWO LINES THAT TURN A HANG INTO AN ERROR MESSAGE

```
if chunk_size <= overlap:
    raise ValueError("overlap must be smaller than chunk_size")
```

RAG or fine-tuning?

They solve different problems. For facts that change, it is not close.

	Fine-tuning	RAG
Where knowledge lives	Baked into the model's weights	In an external store you can edit
Cost to update	A training run, every time	Add a file and re-embed one document
Time to reflect a new CVE	Days to weeks	Minutes
Can it cite a source?	No — the knowledge is diffuse	Yes — you retrieved the passage, so you have it
Can you remove a fact?	Hard. Often needs retraining	Delete the chunk
What it is genuinely good at	Teaching style, format, or a task the model performs badly	Supplying facts the model does not have

The security case is decisive: your facts change weekly, you must be able to cite them, and you occasionally need to remove one. Every one of those favours RAG.

When fine-tuning is right

When the problem is behaviour, not knowledge: a consistent report format, a domain vocabulary, or a task the base model performs poorly.

PART 4

The RAG Recipe & Trust

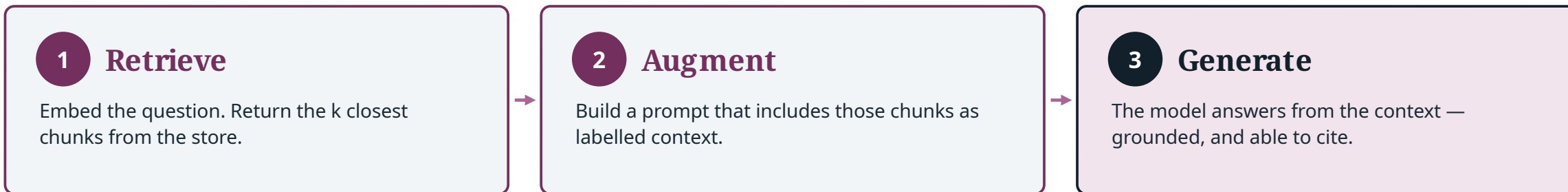
Wire the pieces end to end — and see exactly where the trustworthiness of the answer comes from.

MODULE MAP

- The Problem & the Core Idea
- Embeddings
- Vector DB & Chunking
- **The RAG Recipe & Trust**
- Build It: Notebooks & Practice
- Lab & Mini-Project

The recipe: retrieve, augment, generate

Three moves. The notebook spells them into six steps, but this is the shape.



The whole loop, once the store exists

```

def answer(question, k=4):
    qv = embed_model.encode([question])[0]      # 1 retrieve
    hits = store.query(qv, n_results=k)

    ctx = "\n\n".join(                          # 2 augment
        f"[{i+1}] {d}" for i, d in enumerate(hits["documents"]))

    return llm(SYSTEM, f"Context:\n{ctx}\n\nQuestion: {question}")
                                                    # 3 generate
  
```

Step	In the notebook
1	Load and chunk the documents
2	Embed and store the vectors
3	Retrieve for a question
4	Augment — build the prompt
5	Generate the answer
6	Wrap it all in one function

The grounded prompt is the guardrail

Retrieval supplies the facts. The prompt is what makes the model stay inside them.

```
SYSTEM — the three instructions that do the work

You are a security analyst assistant.

Answer ONLY from the context provided below.
Cite the source of each claim as [n].
If the answer is not in the context, say
    "I don't know based on the provided documents."

Do not use knowledge from outside the context.
```

But notice what you just did. You took text from your corpus and placed it in the prompt, above the question, where the model treats it as instructions-adjacent. If an attacker can write into a document you ingest, they are writing into your prompt. Retrieved text is untrusted input. This is prompt injection, and it is where Module 5 begins.

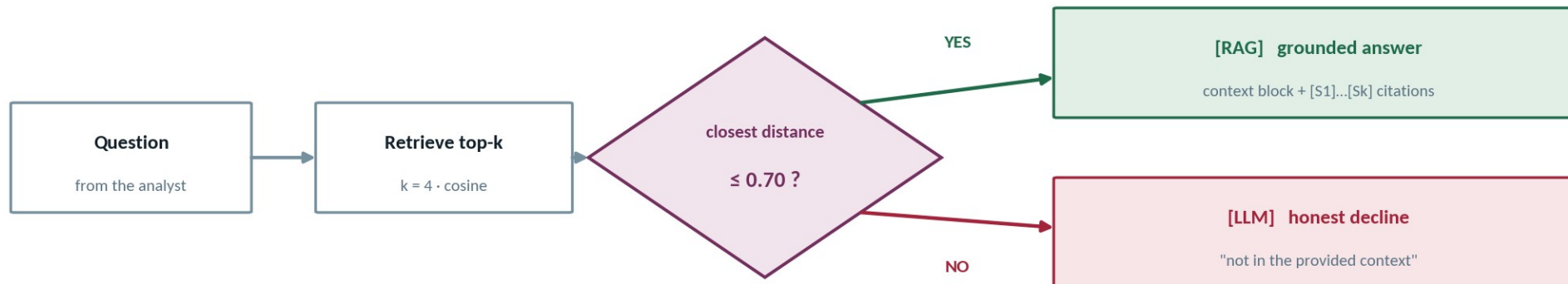
Instruction	What it buys
Answer only from context	Turns generation into summarisation
Cite as [n]	Every claim becomes checkable in seconds
Say "I don't know"	Gives it a licence to fail honestly

The third line is the important one. Without an explicit escape hatch, a model asked an unanswerable question will produce something anyway. "I don't know" has to be permitted.

No API key? Still works. Retrieval runs entirely locally. Generation is the only step that needs one.

The gate: [RAG] or [LLM]

Retrieval does not just supply the context. It decides whether the model is allowed to answer at all.



The test that separates a grounded bot from a plausible one

"What is the CVSS score of Log4Shell?" → distance 0.801 → declines.
The model knows the answer is 10.0 from pre-training.
It must still decline — the fact is true, but it is not grounded.

lecture3_rag_pipeline.ipynb, STEP 3 — THE FILTER IS THE GATE

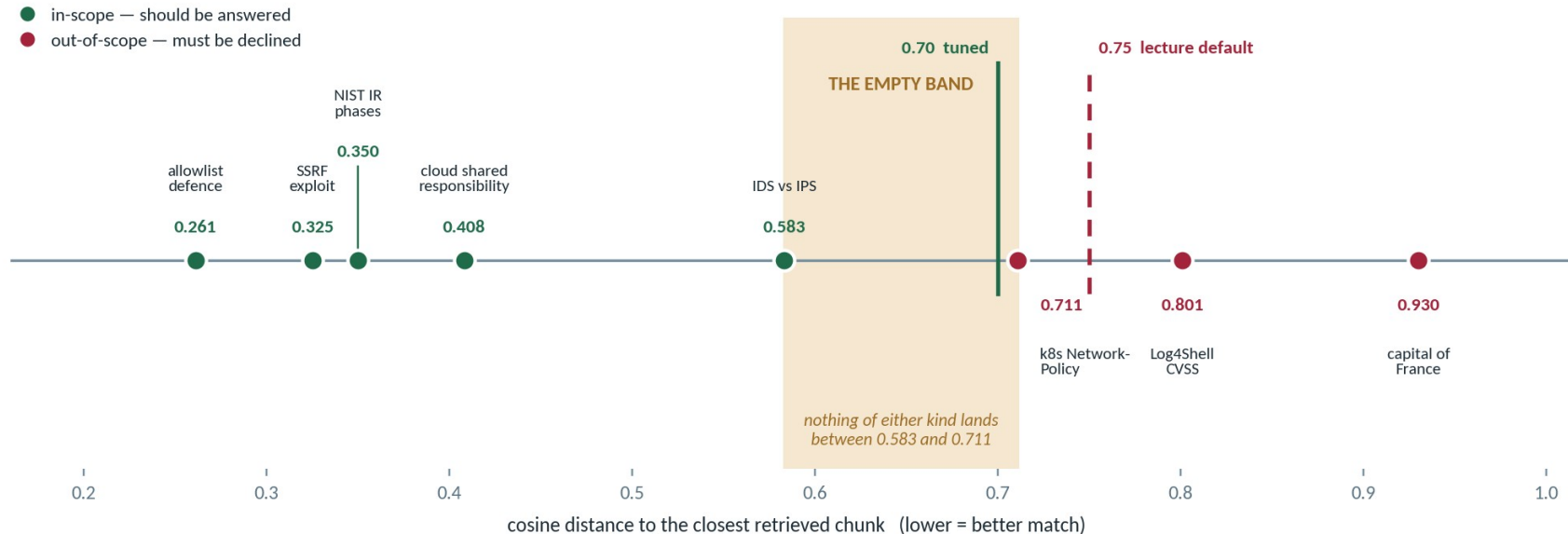
```
def retrieve(query, how_many=4, max_distance=0.75):
    ...
    for k in range(len(found_docs)):
        if found_dists[k] <= max_distance:
            good_hits.append(...) # else: nothing
```

lecture3_rag_pipeline.ipynb, STEP 4 — THE PROMPT ENFORCES IT

```
SYSTEM = (
    "Answer the question using ONLY the context below.\n"
    "If the answer is in the context, prefix [RAG].\n"
    "If the context is empty, say so and prefix [LLM].\n"
    "\n\n# Context\n{context}")
```

Tuning the threshold is not guesswork

Plot the distances you actually measured, and the right cut-off becomes something you can see.



THE FIFTEEN-MINUTE EXPERIMENT

```
for q in in_scope + out_of_scope:
    hits = collection.query(
        query_embeddings=embed([q]), n_results=1)
    print(q, round(hits["distances"][0][0], 3))
```

0.50

4 / 5 answered

too tight — the IDS vs IPS question is wrongly declined

0.70

5 / 5 · 0 leaks

inside the empty band — the tuned recommendation

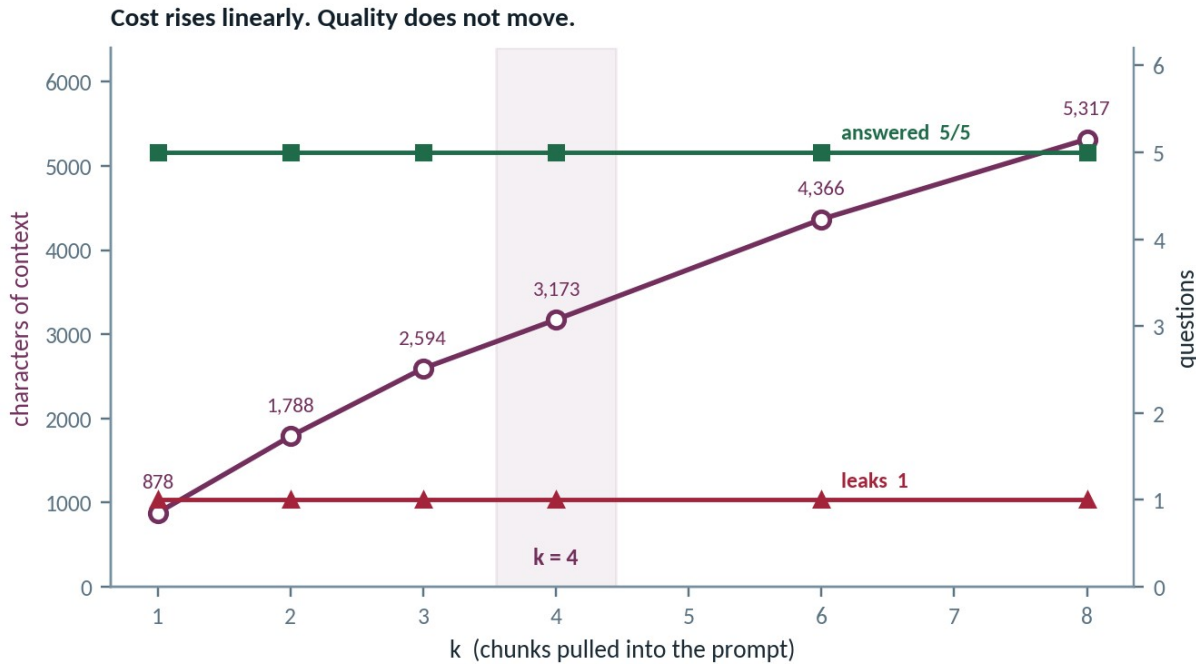
0.75

5 / 5 · 1 leak

the lecture default — answers the Kubernetes question

How much context is enough?

On a corpus this size k is a cost dial, not a quality dial. Say that with numbers rather than a shrug.



```
for k in [1, 2, 3, 4, 6, 8]: hits = retrieve(query, how_many=k)
```

k	context chars	answered	leaks
1	878	5 / 5	1
2	1,788	5 / 5	1
4	3,173	5 / 5	1
8	5,317	5 / 5	1

So why k = 4 and not k = 1? Not recall — k = 1 already answers everything. Corroboration and citation: four passages let the answer cite several sources, and let you see when the top hit was a near miss.

And notice what k never fixes

The single leaked out-of-scope chunk is present at every k. Raising k cannot buy grounding — only the distance threshold does that. Two numbers, two entirely different jobs.

Evaluating a RAG system

"It gave a good answer" is not evaluation. Four checks make it measurable.

1

Retrieval quality

Did the search fetch the chunk that actually answers the question?

2

Grounding

Is every claim supported by a retrieved chunk — or did it drift?

3

Honesty

Does it say "I don't know" when the answer genuinely is not there?

4

Citations

Can a human trace each claim back to a specific source?

Check	How to measure it without a framework	What a failure tells you
Retrieval quality	Write 10 questions with a known answering chunk. Count how often it lands in the top k	Chunking or k is wrong — the generator is not the problem
Grounding	For each claim in the answer, point at the chunk. Anything unpointable is drift	Your prompt is too weak, or too much irrelevant context got in
Honesty	Ask 5 questions the corpus cannot answer. Count real refusals	The "I don't know" instruction is missing or being overridden
Citations	Pick a claim at random. Time how long verification takes	Citations are decorative rather than functional

Diagnose in that order. Retrieval failures masquerade as generation failures, and people tune the prompt for hours when the answering chunk was never retrieved.

PART 5

Build It: The Notebooks & Practice

Two framework-free notebooks over a thousand real CVE descriptions, what RAG is actually used for in security, and the data rules that come with it.

MODULE MAP

- The Problem & the Core Idea
- Embeddings
- Vector DB & Chunking
- The RAG Recipe & Trust
- **Build It: Notebooks & Practice**
- Lab & Mini-Project

The notebooks: framework-free and offline

Nothing hidden. You will see every moving part before you let a framework hide it.

This module's files

```
code/
  lecture2_embeddings_vectordb.ipynb  embeddings + the store
  lecture3_rag_pipeline.ipynb        the full six-step pipeline
data/
  cve_descriptions.csv               1,000 real CVE entries
  kb/                                a small security knowledge base
assignments/                         lab, mini-project, forum
```

Dependency	Why it is the only one
sentence-transformers	Local embeddings, ~80 MB, runs on CPU
chromadb	The store — and you see the NumPy equivalent too
No LangChain	Deliberately. That is next module

Real data, not a toy corpus

A thousand CVE descriptions. Pick one, look at its nearest neighbours, and you will find related vulnerabilities that share no keywords with it at all. That moment is the point of the lab.

Why framework-free: next module you will build this in about fifteen lines of LangChain. You should be able to say what each of those lines is doing — and you can only do that if you have written it yourself once.

Runs offline. Retrieval needs no API key at all.

The ingestion half, line by line

Steps 1 and 2 of lecture3_rag_pipeline.ipynb. This is a script you run once — not a function you call per question.

lecture3_rag_pipeline.ipynb • STEPS 1-2 — INGESTION, RUN ONCE

```
def split_into_chunks(text, chunk_size=600, overlap=100):
    pieces = []
    start = 0
    while start < len(text):
        piece = text[start : start + chunk_size].strip()
        if piece != "":
            pieces.append(piece)
        start = start + chunk_size - overlap
    return pieces

chunks, sources = [], []
for filename in ["ssrf.txt", "cybersecurity_kb.md"]:
    text = open(filename, encoding="utf-8").read()
    file_chunks = split_into_chunks(text)
    for piece in file_chunks:
        chunks.append(piece)          # the chunk text
        sources.append(filename)      # where it came from
    print(filename, "->", len(file_chunks), "chunks")

client = chromadb.EphemeralClient()
collection = client.get_or_create_collection(
    "security_kb", metadata={"hnsw:space": "cosine"})

ids = ["c" + str(i) for i in range(len(chunks))]
labels = [{"source": sources[i]} for i in range(len(chunks))]

collection.add(ids=ids, documents=chunks,
               embeddings=embed(chunks), metadatas=labels)
```

1 strip() and the empty check

Without them the tail of a file produces whitespace-only chunks that match nothing and waste a retrieval slot.

2 Two lists, built in lockstep

chunks and sources stay index-aligned by construction. That alignment is what makes citation possible later.

3 The metric, set explicitly

hnsw:space cosine. Omit it and Chroma uses L2 — every threshold you tuned is then on the wrong scale.

4 embed(chunks) is the expensive line

Every chunk goes through the model. Seconds here, hours on a real corpus — which is why ingestion is a script, not a per-question step.

The query half, line by line

Steps 3 and 4 of lecture3_rag_pipeline.ipynb. This is the function you call on every question — and where grounding is enforced.

lecture3_rag_pipeline.ipynb • STEPS 3–4 — QUERY, RUN EVERY TIME

```
def retrieve(query, how_many=4, max_distance=0.75):
    results = collection.query(query_embeddings=embed([query]),
                               n_results=how_many)

    found_docs = results["documents"][0]
    found_dists = results["distances"][0]
    found_meta = results["metadatas"][0]

    good_hits = []
    for k in range(len(found_docs)):
        if found_dists[k] <= max_distance:      # close enough?
            good_hits.append((found_docs[k],
                               found_dists[k],
                               found_meta[k]))

    return good_hits

SYSTEM = (
    "You are a cybersecurity assistant. Answer the question using "
    "ONLY the context below.\n"
    "If the answer is in the context, prefix your reply with [RAG]. "
    "If the context is empty or does not contain the answer, say so "
    "and prefix with [LLM].\n\n# Context\n{context}"
)

def build_prompt(query, context):
    system_message = SYSTEM.format(
        context=context or "(no relevant context found)")
    return system_message, query
```

1 Same embed() as ingestion

The rule from slide 5. Different models on the two sides and every distance below becomes meaningless.

2 The [0] everywhere

Chroma supports batched queries, so each result list is wrapped. You asked one question, so you take index zero.

3 The filter is the gate

Beyond max_distance, hits are dropped. An out-of-scope question returns an empty list — which is a decision, not a failure.

4 The "or" in build_prompt

An empty context becomes an explicit “no relevant context found” rather than a blank. Models improvise into blanks.

The whole bot, in one function

Step 6 of lecture3_rag_pipeline.ipynb. Retrieve, augment, generate — and two calls that prove it can decline.

lecture3_rag_pipeline.ipynb · STEP 6 — THE COMPLETE PIPELINE

```
# the complete RAG function: retrieve -> augment -> generate
def ask(query):
    hits = retrieve(query)

    # join the retrieved chunks into one context block
    context_pieces = []
    for doc, dist, meta in hits:
        context_pieces.append(doc)
    context = "\n\n".join(context_pieces)

    system, user = build_prompt(query, context)

    print("Q:", query)
    if len(hits) > 0:
        print(" grounded? YES (found relevant context)")
    else:
        print(" grounded? NO (no relevant context)")
    print(" answer:", generate(system, user)[:300], "\n")

ask("What is SSRF and how can it be exploited?")
# -> in the documents -> [RAG]

ask("What is the capital of France?")
# -> not in the documents -> [LLM] fallback
```

1 Three moves, nothing else

retrieve, join into context, build prompt, generate. The recipe from Part 4, now executable.

2 Keep the grounded flag

Every answer arrives labelled with whether it had evidence. You can audit a transcript at a glance.

3 Two calls, not one

One question the corpus answers, one it cannot. That pair is the acceptance test for your mini-project.

4 The model knows it is Paris

And must still decline. A true fact that is not grounded is exactly the failure this architecture exists to prevent.

What RAG is actually used for in security

Four patterns, all of which are somebody's real internal tool.

Query threat intel

Ask advisories and intel reports in natural language instead of grepping a PDF pile.

Answer policy questions

"What does NIST AI RMF say about model monitoring?" — with a citation you can check.

Search incident history

During response: has anything like this happened before, and what did we do?

Ground a SOC assistant

Answers that reflect your runbooks and your environment, not the open internet.

Pattern	The corpus	The property that makes it work
Threat intel Q&A	Advisories, vendor bulletins, intel reports	Currency — add today's advisory and it is searchable now
Policy and framework Q&A	NIST, ISO, internal policy documents	Citation — an uncited compliance answer is worthless
Incident knowledge base	Past tickets, post-incident reviews	Semantic match — nobody describes an incident the same way twice
SOC assistant	Your runbooks and asset documentation	Privacy — local embeddings mean the corpus never leaves

Notice the third column. Each pattern depends on a different property of RAG — which is why "just use RAG" is not a design.

Beyond one-shot retrieval

Standard RAG retrieves once and answers. An agent decides what to retrieve, looks at what came back, and tries again.

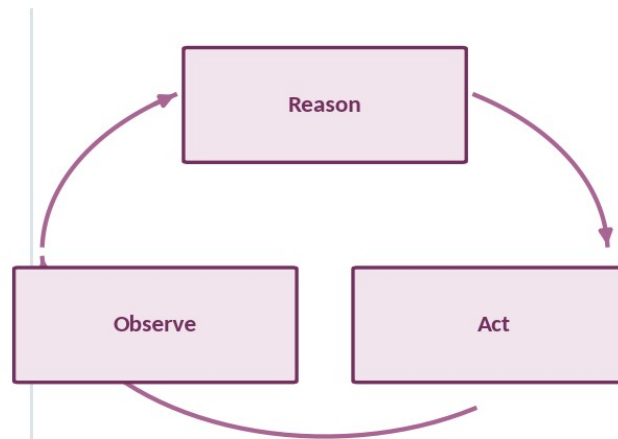
STANDARD RAG • a linear pipeline



"an expert librarian finding a specific book"

one pass • no memory of what it just read

AGENTIC RAG • a reason-act-observe loop



- decompose a multi-part request
- pick the right tool per sub-task
- re-query when results are ambiguous
- synthesise across several sources

"an entire research team investigating a topic"

"Agentic RAG transforms the process into a reason-act-observe loop." The agent reasons about the goal, acts by using its tools, and observes the results to inform its next step. This allows it to handle ambiguity and tackle multifaceted problems. — Santos, *Agentic AI for Cybersecurity*, Ch. 2

Where this goes next

Module 5 gives you the framework, Module 6 gives you the loop. The `retrieve()` you wrote this week becomes one of the tools the agent is allowed to call — nothing here is thrown away.

Data handling: retrieved text is your prompt

Everything in your corpus can end up in an answer. Plan for that before you ingest.

Rule	Concretely	Why it bites in RAG specifically
Embed locally for sensitive data	sentence-transformers on your machine, not an embedding API	Ingestion sends every chunk of the corpus, not just one prompt
Sanitise the corpus first	Strip credentials, keys, PII and customer names before ingesting	Retrieval will surface whatever is in a chunk, to whoever asks
Treat retrieved text as untrusted	It reaches the model in an instruction-adjacent position	Anyone who can write into an ingested document writes into your prompt
Labs use provided documents only	Synthetic or course-supplied corpora — never real incident data	A corpus is a bigger disclosure than a single prompt
Keys in .env or Colab Secrets	Never in a cell, never in the report	Unchanged from Module 2, and still the most common slip

The one that surprises people: row two. A RAG system has no concept of who is asking. If a runbook chunk contains a service-account password, then anyone who can query the assistant can retrieve that password by asking a question that happens to match. Access control has to live in the metadata filter and in what you chose to ingest — never in the model's discretion.

PART 6

Lab & Mini-Project

A concept-check lab to rehearse the machinery, then your own working RAG Q&A bot with a tuning experiment attached.

MODULE MAP

-  The Problem & the Core Idea
-  Embeddings
-  Vector DB & Chunking
-  The RAG Recipe & Trust
-  Build It: Notebooks & Practice
-  **Lab & Mini-Project**

The concept-check lab — your rehearsal

Run the notebooks, poke at them, and confirm the machinery makes sense before you build.

	Do this	You should be able to say
1	Run both notebooks top to bottom	Which cell does ingestion and which does query
2	Pick a CVE, read its nearest neighbours	Why a result with no shared words came back
3	Change the chunk size and re-run	What happened to the number of chunks, and why
4	Ask something the corpus cannot answer	Whether it refused honestly, and what made it refuse
5	Print the retrieved chunks before the answer	Whether a bad answer was retrieval's fault or generation's

The four ideas it checks

Embeddings · cosine similarity · chunking · the three RAG steps.

If any of those still feels vague after the lab, sort it out before starting the mini-project rather than during it.

Why a rehearsal at all: the mini-project should be assembly, not discovery. Every component you need is in these notebooks. If you have run them and understood them, building your own bot is a couple of hours of wiring. If you have not, it is an evening of debugging your understanding and your code simultaneously, which is much harder than doing either alone.

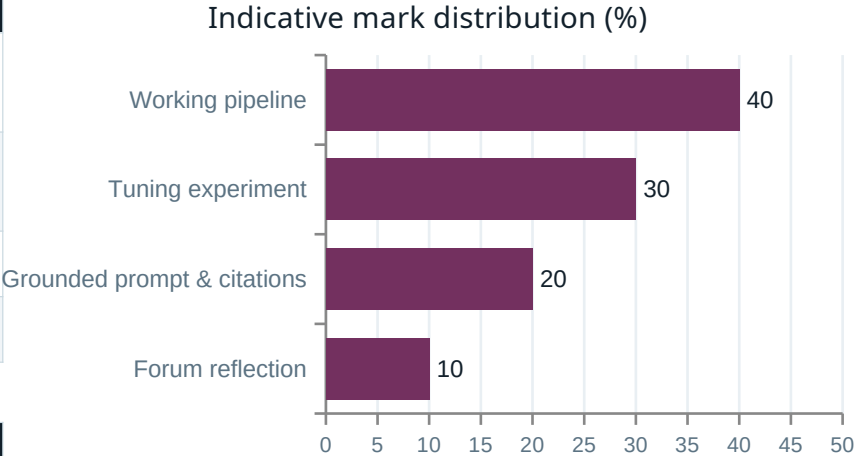
Step 5 is the one to internalise: printing the retrieved chunks before the answer is the single most useful debugging habit in RAG. It tells you immediately which half of the system is broken, and it will save you hours on the mini-project.

Mini-project: build a RAG Q&A bot

Your own framework-free pipeline over your own documents — plus the experiment that proves you understand it.

	Deliverable	What I am looking for
1	A small knowledge base	2–3 security documents, or the course KB. Say why you chose them
2	A runnable notebook	Ingest → retrieve → augment → generate, with the phases separated
3	A tuning experiment	Before and after numbers, not impressions
4	A forum reflection	On grounding and trust, in your own words

Knob	Try	Report the effect on
chunk_size	256 → 512 → 1024	Whether the answering passage still lands in the top k
k	2 → 4 → 8	Answer quality, and how much irrelevant context got in
distance threshold	Loose → strict	How often it correctly says "I don't know"



The tuning experiment is 30%. A bot that answers questions is the expected outcome. A student who can explain what changed when they moved chunk size from 256 to 1024 — and why — has understood the module.

Provided or synthetic documents only — no real incident data, no credentials.

Four traps that cost marks

Every one of these has cost a student an afternoon. Not one of them raises an error.

get_or_create + reused ids · the silent failure

```
kb = client.get_or_create_collection("kb")
kb.add(ids=["c0","c1",...]) # ids already exist
```

size	chunks	distance
600	24	0.325
1000	24	0.325
1600	24	0.325

every row identical — the new chunks were never stored

delete first · what the rubric expects

```
client.delete_collection("kb")
kb = client.create_collection("kb", metadata={...})
```

size	chunks	distance
600	24	0.325
1000	19	0.341
1600	10	0.622

real variation — this is the 15-pt tuning experiment

2 · Chroma defaults to L2

Without metadata={"hnsw:space": "cosine"} your distances are on a different scale and every threshold you tuned is wrong.

3 · overlap ≥ chunk_size

The stride goes to zero and the splitter loops forever. No exception, no output — just a cell that never finishes.

4 · relative paths and the first run

Run from RAG/code/. The first model load pulls ~90 MB and looks frozen; pass device="cpu" if the GPU driver mismatches.

Recap and what's next

Four things to carry into the framework modules.

1 Meaning becomes geometry

Embeddings map text to vectors; cosine similarity finds related passages that share no words.

2 Two phases, not one

Ingest once — chunk, embed, store. Query many times — retrieve, augment, generate.

And the uncomfortable one

You built a system that takes text from a corpus and puts it into a prompt. Retrieved text is untrusted input, and every retrieval system is an injection surface.

3 The prompt is the guardrail

Answer only from context, cite [n], and permit "I don't know." That last one does the most work.

4 Diagnose retrieval first

Print the chunks before you tune the prompt. Retrieval failures look exactly like generation failures.

Next — LangChain: the same pipeline in about fifteen lines, so you can see what a framework buys you — and then we go straight at prompt injection.